

Woche 3: Datenstrukturen

Einleitung

Willkommen zur dritten Woche Ihres Python-Selbststudiums! Sie haben bereits die Grundlagen der Programmierung und von Python gelernt und sind bereit, tiefer in die faszinierende Welt der Programmierung einzutauchen. In dieser Woche werden wir uns auf Datenstrukturen konzentrieren, eine Schlüsselkomponente jeder Programmiersprache und besonders wichtig in Python.

In dieser Woche werden wir uns auf die verschiedenen Arten von Datenstrukturen konzentrieren, die in Python verfügbar sind, einschließlich Strings, Listen, Tupeln, Mengen und Dictionaries. Sie werden lernen, wie man diese Strukturen erstellt, manipuliert und in andere Formate umwandelt. Sie werden auch lernen, wie man mit verschachtelten Strukturen arbeitet, was eine starke Fähigkeit in der Welt der Programmierung ist.

Ihre Aufgaben für diese Woche konzentrieren sich auf die praktische Anwendung dieser Konzepte.

Sie werden Datenstrukturen manipulieren, sie in verschiedene Formate umwandeln und mit verschachtelten Strukturen arbeiten. Am Ende der Woche sollten Sie sich wohl fühlen, diese Konzepte in Ihren eigenen Projekten anzuwenden.

Am Übungstag werden wir das Gelernte wiederholen und vertiefen, um sicherzustellen, dass Sie ein solides Verständnis der Materie haben. Wir werden auch in die List-, Dictionary- und SetComprehensions eintauchen, eine mächtige und effiziente Methode, um mit Datenstrukturen in Python zu arbeiten.

Vor uns liegt also eine spannende Woche voller neuer Lernmöglichkeiten. Mit jedem Schritt auf dieser Reise werden Sie sich weiter als Python-Programmierer entwickeln. Bleiben Sie dran und lassen Sie uns diese Woche mit Begeisterung angehen!

Gesamtüberblick

Hier ein Überblick über die Inhalte und Aktivitäten der aktuellen Woche:

- Selbststudium:
 - Strings
 - Listen
 - Tupel
 - Mengen
 - Listenoperationen
 - Dictionaries
 - Dictionary-Operationen
- Aufgabe:
 - Manipulation von Datenstrukturen
 - Umwandlung Liste zu Tupel, Liste zu Set, String zu Liste, Liste zu String
 - Umwandlung Liste zu Dictionary und Dictionary zu Liste
 - Suche und Manipulation der Daten
 - Verschachtelte Strukturen
- Übungstag 3:
 - Wiederholung
 - Vertiefung: Anlage und Bearbeitung
 - Vertiefung: verschachtelte Strukturen
 - Ergänzung: List-/Dict-/Set-Comprehensions
 - Ausblick: Funktionen und Module

Inhalte und thematische Abgrenzung

Die folgende Auflistung zeigt detailliert, welche Themen Sie in der Woche behandeln und bearbeiten. Sie sind eine Voraussetzung für die folgenden Wochen und sollten gut verstanden worden sein. Wenn es Verständnisprobleme gibt, machen Sie sich Notizen und fragen Sie am Präsenztage nach, so dass wir gemeinsam zu Lösungen kommen können. Und denken Sie bitte immer daran: es gibt keine „dummen“ Fragen!

- Strings
 - Erstellung und Zuweisung von Strings
 - Escape-Sequenzen in Strings
 - String-Methoden (z.B. upper(), lower(), strip(), split(), join(), etc.)
 - Formatierung von Strings (f-strings)
 - Indexierung und Slicing von Strings
 - Unveränderlichkeit von Strings
- Listen
 - Erstellen von Listen
 - Zugriff auf Listenelemente
 - Hinzufügen und Entfernen von Elementen
 - Slicing von Listen
 - Listen durchlaufen
- Tupel
 - Erstellen von Tupeln
 - Zugriff auf Tuplelemente
 - Unveränderlichkeit von Tupeln
 - Tupel durchlaufen
- Mengen
 - Erstellen von Mengen
 - Hinzufügen und Entfernen von Elementen in Mengen
 - Set-Operationen (Vereinigung, Schnittmenge, Differenz)
- Listenoperationen
 - Listen sortieren
 - Listen umkehren und Anzahl der Elemente in einer Liste finden
 - Prüfung, ob ein Element in einer Liste vorhanden ist
- Dictionaries
 - Erstellen von Dictionaries
 - Zugriff auf Dictionary-Elemente
 - Hinzufügen und Entfernen von Elementen in einem Dictionary
 - Durchlaufen eines Dictionaries
- Dictionary-Operationen
 - Überprüfen, ob ein Schlüssel in einem Dictionary vorhanden ist
 - Abrufen aller Schlüssel und Werte aus einem Dictionary
 - Verschachtelte Dictionaries

Lernpfad

Der Lernpfad ist ein Vorschlag, in welcher Reihenfolge Sie die Inhalte der Woche angehen können.

Betrachten Sie ihn gerne als eine Todo-Liste, die Sie von oben nach unten abhaken. So können Sie

sicher sein, dass Sie alle wichtigen Themen bearbeitet haben und sind gut vorbereitet für die folgenden Wochen.

1. Vorbereitung

- Installation zusätzlicher Python-Bibliotheken (falls erforderlich)
- Bereitstellung empfohlener Lernmaterialien (Bücher, Online-Kurse, Videos)

2. Strings

- Verständnis der Struktur und Eigenschaften von Strings
- Manipulation von Strings (Zugriff auf Zeichen, Slicing, Methoden)
- Umwandlung zwischen Strings und Listen

3. Listen

- Verstehen, wie Listen in Python funktionieren
- Manipulation von Listen (Hinzufügen/Entfernen von Elementen, Slicing, Methoden)
- Umwandlung zwischen Listen und anderen Datentypen (Tupel, Mengen)

4. Tupel

- Verständnis der Eigenschaften und Verwendungszwecke von Tupeln
- Manipulation von Tupeln (Zugriff auf Elemente, Slicing, Methoden)
- Umwandlung zwischen Tupeln und Listen

5. Mengen

- Verständnis der Eigenschaften und Verwendungszwecke von Mengen
- Durchführung von Mengenoperationen (Vereinigung, Schnittmenge, Differenz)
- Umwandlung zwischen Mengen und Listen

6. Listenoperationen

- Durchführung verschiedener Operationen auf Listen (Sortierung, Suche, Aggregation)
- Anwendung von List Comprehensions zur effizienten Manipulation von Listen

7. Dictionaries

- Verstehen, wie Dictionaries in Python funktionieren
- Manipulation von Dictionaries (Hinzufügen/Entfernen von Schlüssel-Wert-Paaren, Zugriff auf Werte, Methoden)
- Umwandlung zwischen Dictionaries und Listen

8. Dictionary-Operationen

- Durchführung verschiedener Operationen auf Dictionaries (Sortierung, Suche, Aggregation)
- Anwendung von Dictionary Comprehensions zur effizienten Manipulation von Dictionaries

9. Übungsaufgaben

- Manipulation von Datenstrukturen
- Umwandlung zwischen verschiedenen Datentypen
- Suche und Manipulation der Daten
- Arbeit mit verschachtelten Strukturen

10. Selbstbewertung

- Durchführung von Multiple-Choice-Bewertungen
- Überprüfung der wichtigsten Konzepte und Fähigkeiten
- Nachbearbeitung von schwierigen Themen

11. Online-Schulungstag

- Wiederholung der gelernten Themen
- Vertiefung der Arbeit mit Datenstrukturen
- Einführung in List-/Dict-/Set-Comprehensions
- Ausblick auf Funktionen und Module

Programmieraufgaben

Die folgenden Programmieraufgaben sollen Ihnen eine Anregung geben. Haben Sie eigene Ideen und Themen, die Sie ausprobieren wollen, dann sollten Sie diesen nachgehen. Wichtig ist vor allem, dass Sie „Dinge ausprobieren“. Und auch, dass Sie Fehler machen, sowohl syntaktische als auch semantische. Versuchen Sie diese Fehler zu finden und aufzulösen, dann gerade aus den Fehlern lernen Sie am Ende am meisten.

- Strings:
 - Schreiben Sie ein Python-Programm, um einen eingegebenen String umzudrehen.
 - Schreiben Sie ein Python-Programm, das alle Vokale in einem gegebenen String zählt.
 - Schreiben Sie ein Python-Programm, das einen String in eine Liste von Wörtern umwandelt, und dann die Liste wieder in einen String umwandelt.
- Listen:
 - Schreiben Sie ein Python-Programm, das die Elemente einer gegebenen Liste in umgekehrter Reihenfolge ausgibt.
 - Schreiben Sie ein Python-Programm, das das größte und kleinste Element in einer Liste findet.
 - Schreiben Sie ein Python-Programm, das alle Duplikate aus einer Liste entfernt.
- Tupel:
 - Schreiben Sie ein Python-Programm, das ein Tupel in eine Liste umwandelt und umgekehrt.
 - Schreiben Sie ein Python-Programm, das das n-te Element eines gegebenen Tupels findet.
 - Schreiben Sie ein Python-Programm, das überprüft, ob ein gegebenes Element in einem Tupel vorhanden ist.
- Mengen:
 - Schreiben Sie ein Python-Programm, das die Vereinigung und Überschneidung zweier Mengen berechnet.
 - Schreiben Sie ein Python-Programm, das überprüft, ob eine Menge eine Teilmenge einer anderen ist.
 - Schreiben Sie ein Python-Programm, das eine Liste in eine Menge umwandelt und alle Duplikate entfernt.
- Listenoperationen:
 - Schreiben Sie ein Python-Programm, das die Summe aller Zahlen in einer gegebenen Liste berechnet.
- Dictionaries:
 - Schreiben Sie ein Python-Programm, das ein Dictionary erstellt, das Zahlen von 1 bis n und ihre Quadrate enthält (n ist eine Eingabe des Benutzers).
 - Schreiben Sie ein Python-Programm, das alle Schlüssel eines gegebenen Dictionary ausgibt.

- Schreiben Sie ein Python-Programm, das ein Dictionary von einer Liste von Schlüsseln und einer Liste von Werten erstellt.
- Dictionary-Operationen:
 - Schreiben Sie ein Python-Programm, das die Summe aller Werte in einem gegebenen Dictionary berechnet.
 - Schreiben Sie ein Python-Programm, das ein Dictionary in eine Liste von Tupeln umwandelt (jedes Tupel besteht aus einem Schlüssel und seinem zugehörigen Wert).
 - Schreiben Sie ein Python-Programm, das überprüft, ob ein gegebener Schlüssel in einem Dictionary vorhanden ist.

Abschluss-Quiz

Das Quiz soll Ihnen einen ersten Hinweis auf Ihren Lernfortschritt geben. Nach unserer Einschätzung sollten Sie diese Fragen alle beantworten können, wenn Sie den Stoff der Woche durchgearbeitet und verstanden haben. Natürlich gibt es noch sehr viel mehr mögliche Fragen, dazu wollen wir auf

die Literatur und das Internet verweisen. Geben Sie gerne einmal „python quizzes“ bei Google ein.

1. Welcher der folgenden Datentypen in Python ist unveränderlich?
 - a) List
 - b) Dictionary
 - c) Set
 - d) Tuple
2. Wie greifen Sie auf das letzte Element einer Liste namens 'my_list' zu?
 - a) my_list[0]
 - b) my_list[-1]
 - c) my_list[end]
 - d) my_list[1]
3. Wie erstellen Sie eine neue Menge in Python?
 - a) set = {1, 2, 3}
 - b) set = Set(1, 2, 3)
 - c) set = [1, 2, 3]
 - d) set = (1, 2, 3)
4. Wie entfernen Sie ein Element aus einem Dictionary?
 - a) del dict[element]
 - b) dict.remove(element)
 - c) dict.pop(element)
 - d) dict.delete(element)
5. Wie fügen Sie ein Element zu einer Liste hinzu?
 - a) list.add(element)
 - b) list.insert(element)
 - c) list.extend(element)
 - d) list.append(element)
6. Welche Methode wird verwendet, um ein Element aus einer Liste zu entfernen?
 - a) remove()
 - b) pop()
 - c) delete()
 - d) discard()
7. Wie erstellen Sie ein Dictionary, das Schlüssel-Wert-Paare speichert?

- a) {'key': 'value'}
- b) ('key', 'value')
- c) {key: value}
- d) ["key", "value"]

8. Wie überprüfen Sie, ob ein Schlüssel in einem Dictionary vorhanden ist?

- a) 'key' in dict
- b) dict.has_key('key')
- c) dict.contains('key')
- d) 'key' in dict.keys()

9. Wie erstellen Sie ein leeres Tuple in Python?

- a) tuple = ()
- b) tuple = []
- c) tuple = {}
- d) tuple = empty()

10. Wie überprüfen Sie, ob ein Element in einer Menge enthalten ist?

- a) set.contains(element)
- b) element in set
- c) set.has(element)
- d) element is set